

IT3280 – THỰC HÀNH KIẾN TRÚC MÁY TÍNH

BÁO CÁO THỰC HÀNH GIỮA KÌ

Họ và tên: Nguyễn Hồng Khôi

MSSV: 202416956

Assignment 1: Bài 8 phần A số nguyên.

.data

prompt: .asciz "Nhap so nguyen duong N: "

res_msg: .asciz "Chu so lon nhat la: "

buffer: .space 100

.text

main:

#nhap chuoi N

li a7, 4

la a0, prompt

ecall

li a7, 8 # doc chuoi

la a0, buffer # Dia chi bo nho dem

li a1, 100 # Do dai toi da

ecall

#khai tao

la t0, buffer # t0 tro den ky tu dau tien

lb t1, 0(t0) # t1 giu chu so lon nhat hien tai (tam thoi la ki tu dau tien)

kiem tra rong

beqz t1, end_program

loop:

addi t0, t0, 1 # Chuyen den ky tu tiep theo

lb t2, 0(t0) # Doc ky tu hien tai vao t2

```
# kiem tra ket thuc
```

```
li t3, 10      # ktra ki tu xuong dong, neu la \n thi end chuoi
```

```
beq t2, t3, print_result
```

```
beqz t2, print_result
```

```
#so sanh
```

```
ble t2, t1, loop  # Neu t2 <= t1 thi tiep tục lap
```

```
mv t1, t2      # Neu t2 > t1 thi cap nhat t1 = t2
```

```
j loop
```

```
print_result:
```

```
#kqua
```

```
li a7, 4      # In thong bao ket qua
```

```
la a0, res_msg
```

```
ecall
```

```
li a7, 11     # In ki tu max
```

```
mv a0, t1
```

```
ecall
```

```
end_program:
```

```
li a7, 10
```

ecall

- **Thanh ghi t0** :Giữ địa chỉ của lần lượt các ký tự số N. Ban đầu nó trỏ vào địa chỉ đầu tiên của buffer. Sau mỗi lần lặp, addi t0, t0, 1 để dịch chuyển sang địa chỉ của ký tự tiếp theo.
- **Thanh ghi t1 (Chữ số lớn nhất tạm thời):**
 - Lúc đầu, nó nhận giá trị của ký tự đầu tiên (lb t1, 0(t0)).
 - Trong quá trình lặp, nếu tìm thấy một ký tự (t2) lớn hơn t1, giá trị của t2 sẽ được copy sang t1 thông qua lệnh mv t1, t2.
- **Thanh ghi t2 (Ký tự hiện tại):** Dùng để load giá trị tại vị trí con trỏ t0 đang đứng để thực hiện phép so sánh với t1.
- **Thanh ghi a0:** Dùng làm tham số đầu vào cho các lệnh gọi hệ thống (lưu địa chỉ chuỗi hoặc giá trị ký tự cần

Ví dụ: N = 25

*Bước khởi tạo và nhập dữ liệu:

-Ban đầu chưa được gán giá trị:

t0	5	0x00000000
t1	6	0x00000000
t2	7	0x00000000

-Nhập N=25:

```
-- program is finished running (0) --  
Nhap so nguyen duong N: 25
```

-Thanh ghi t0 trỏ vào vị trí đầu tiên của buffer:

t0	5	268501028
----	---	-----------

*Vòng lặp:

Lần 1:

-Vòng đầu mặc định gán t1 là số lớn nhất nên ch gán t2, t2 được gán từ kí tự thứ 2.

t2	7	0
----	---	---

-Thanh ghi t1 giữ kí tự 2(mã ascii là 50):

t1	6	50
----	---	----

Lần 2:

-Thanh ghi t2 giữ kí tự 5(ascii 53):

t2	7	53
----	---	----

-Thanh ghi t1 giữ kí tự 5(do $5 > 2$):

t1	6	53
----	---	----

Lần 3:

-Thanh ghi t0 ở địa chỉ kí tự tiếp theo

t0	5	268501041
----	---	-----------

-Do hết chữ số 25 nên t2 trở về kí tự xuống dòng:

t2	7	0x0000000a
----	---	------------

=>Chương trình kết thúc in ra chữ số lớn nhất của số 25 là 5:

```
nhập số nguyên dương n: 456
Nhập số nguyên dương N: 25
Chu số lớn nhất là: 5
```

Assignment 2: Bài 4 phần B: Mảng.

.data

prompt_n: .asciz "Nhap so luong phan tu n: "

prompt_val: .asciz "Nhap phan tu: "

res_msg: .asciz "Cap phan tu lien ke co tong nho nhat la: "

and_msg: .asciz " va "

array: .word 0:100

.text

main:

#nhap so luong phan tu n

li a7, 4 # In thông báo nhập n

la a0, prompt_n

ecall

li a7, 5 # Đọc số nguyên n

ecall

mv s0, a0 # s0 = n (gán số phần tử vào s0)

#nhap mang

la s1, array # s1 = địa chỉ base của mảng

```
li t0, 0      # t0 = i (bien dem)
```

```
input_loop:
```

```
    bge t0, s0, process # Neu i >= n thi chuyen sang xu ly
```

```
    li a7, 4      # tba0 nhap ptu mang
```

```
    la a0, prompt_val
```

```
    ecall
```

```
    li a7, 5      # Doc so nguyen
```

```
    ecall
```

```
    slli t1, t0, 2  # t1 = i * 4(moi so nguyen 4 byte)
```

```
    add t1, s1, t1  # t1 = dia chi mang[i]
```

```
    sw a0, 0(t1)    # Luu gia tri vao mang
```

```
    addi t0, t0, 1  # i++
```

```
    j input_loop
```

```
process:
```

```
    #Tim cap co tong nho nhat
```

```
    # Khoi tao: lay cap 2 ptu dau tien cua mang la a[0] va a[1] bat dau chayj thuat toan
```

```
    lw t2, 0(s1)    # t2 = array[0]
```

```
    lw t3, 4(s1)    # t3 = array[1]
```

add t4, t2, t3 # t4 = tong nho nhat hien tai (minSum)

mv s2, t2 # s2 = so thu nhat cua cap min

mv s3, t3 # s3 = so thu hai cua cap min(s2,s3 luu gia tri cap nho nhat khi do)

li t0, 1 # t0 = i (bat dau tu phan tu thu 2 de xet cap i, i+1)

addi t5, s0, -1 # end vong

find_min_loop:

bge t0, t5, print_result

slli t1, t0, 2 # t1 = i * 4

add t1, s1, t1 # t1 = dia chi array[i]

lw t2, 0(t1) # t2 = a[i]

lw t3, 4(t1) # t3 = a[i+1]

add t6, t2, t3 # t6 = tong hien tai

bge t6, t4, next_iter # Neu t6tong dg xet) >= t4(min sum hien tai) thi bo qua

mv t4, t6 # Cap nhat tong min moi

mv s2, t2 # Cap nhat phan tu 1

mv s3, t3 # Cap nhat phan tu 2

next_iter:

addi t0, t0, 1 # i++

j find_min_loop

print_result:

in ket qua

li a7, 4

la a0, res_msg

ecall

li a7, 1 # In so thu nhat (s2)

mv a0, s2

ecall

li a7, 4 # In chu "va"

la a0, and_msg

ecall

li a7, 1 # In so thu hai (s3)

mv a0, s3

ecall

li a7, 10

ecall

Mô tả thuật toán sử dụng:

-Thuật toán Cửa sổ trượt (Sliding Window):

Thuật toán tìm cặp liên tiếp có tổng nhỏ nhất hoạt động theo cơ chế "cửa sổ" kích thước bằng 2:

- **Bước 1:** Khởi tạo "cửa sổ" tại vị trí a[0] và a[1]. Tính tổng và tạm gọi là minSum.
- **Bước 2:** Trượt cửa sổ sang phải 1 đơn vị (xét a[1] và a[2]).
- **Bước 3:** Tại mỗi vị trí, nạp dữ liệu từ RAM vào thanh ghi t2 (A[i]) và t3 (A[i+1]) thông qua lệnh lw (Load Word).
- **Bước 4:** So sánh tổng mới với minSum. Nếu nhỏ hơn, "ghi đè" giá trị mới vào thanh ghi lưu trữ kết quả.

*Khởi tạo:

-Giá trị ban đầu của các thanh ghi:

t0	5	0x00000000
t1	6	0x00000000
t2	7	0x00000000
s0	8	0x00000000
s1	9	0x00000000
a0	10	0x00000000
a1	11	0x00000000
a2	12	0x00000000
a3	13	0x00000000

-Sử dụng các giá trị khởi tạo là: n=6

Array = [3, 6, -2, -5, 7, 3]

```

jes Run I/O
Nhap so luong phan tu n: 6
Nhap phan tu: 3
Nhap phan tu: 6
Nhap phan tu: -2
Nhap phan tu: -5
Nhap phan tu: 7
Nhap phan tu: 3

```

-Địa chỉ gốc s1 của mảng:

s1	9	0x10010058
----	---	------------

*Vòng lặp:

- Trong vòng lặp sẽ liên tục cập nhật t4(tổng nhỏ nhất hiện tại) và t6(tổng tại vòng lặp đó) cho đến khi đi qua hết phần tử của mảng ta thu được t4 và vị trí s2,s3 là vị trí 2 số liên tiếp có tổng nhỏ nhất:

-Ban đầu 2 thanh chưa được gán giá trị:

t4	29	0x00000000
t5	30	0x00000000
t6	31	0x00000000

-Vòng 1:

-thanh ghi t2,t3 ở 2 phân tử đầu và tổng t4(min hiện tại):

t2	7	0x00000003
t3	28	0x00000006
t4	29	0x00000009

-do hiện tại a[0] và a[1] là nhỏ nhất nên cập nhật s2,s3. Các vòng lặp sau đó cập nhật vị trí tương tự:

s2	18	0x00000003
s3	19	0x00000006

-Vòng 2:

-trượt đến vị trí tiếp theo là a[1] và a[2] (số âm):

t2	7	0x00000006
t3	28	0xffffffffe

-tổng hiện tại t6:

t6	31	0x00000004
----	----	------------

-do $t4 > t6$ nên cập nhật lại $t4 = t6, s2 = t2, s3 = t3$ mới:

t4	29	0x00000004
a7	17	0x00000005
s2	18	0x00000006
s3	19	0xffffffffe

-Vòng 3:

-trượt đến vị trí a[2] và a[3]:

t1	6	0x10010000
t2	7	0xffffffffe
t3	28	0xffffffffb

-tổng t6:

t6	31	0xffffffff9
----	----	-------------

- $t4 > t6$ nên cập nhật lại $t4$ và $s2, s3$:

t4	29	0xffffffff9
s2	18	0xffffffffe
s3	19	0xffffffffb

-Vòng 4:

-trượt đến vị trí a[3] và a[4]:

t2	7	0xffffffffb
s11	27	0x00000000
t3	28	0x00000007

-tổng hiện tại $t6 > t4$ nên bỏ qua:

t6	31	0x00000002
----	----	------------

-Vòng 5:

-trượt đến a[4], a[5]:

t2	7	0x00000007
----	---	------------

t3	28	0x00000003
----	----	------------

-tổng $t6 > t4$ nên bỏ qua:

t6	31	0x00000002
----	----	------------

-Vòng 6:

-trượt đến a[5], a[6]:

-kết thúc vòng lặp do đã ở phần tử cuối, thanh ghi s0 đã ở cuối array:

s0	8	0x00000006
----	---	------------

-kết quả in ra:

```
Nhap phan tu: 6
Nhap phan tu: -2
Nhap phan tu: -5
Nhap phan tu: 7
Nhap phan tu: 3
Cap phan tu lien ke co tong nho nhat la: -2 va -5
-- program is finished running (0) --
```

Assignment3: b11_C_Chui:

.data

prompt: .asciz "Nhap xau ky tu: "

res_msg: .asciz "Ket qua sau khi doi: "

buffer: .space 100

.text

main:

 # nhap xau ky tu

 li a7, 4 # in tb

 la a0, prompt

 ecall

 li a7, 8 # nhap xau tu ban phim

 la a0, buffer

 li a1, 100

 ecall

 #duyet tung ky tu vakiem tra bien doi

 la t0, buffer # t0 trở vào buffer

loop:

 lb t1, 0(t0) # load ky tu hien tai vao t1

 #kiem tra ket thuc chuoi(\n)

 beqz t1, print_result

 li t2, 10

 beq t1, t2, print_result

```
# kiem tra neu la chu HOA (ma ascii la 65 - 90)
```

```
li t2, 65      # A
```

```
li t3, 90      # Z
```

```
blt t1, t2, check_lower # neu < 65 co the la chu thuong hoac ky tu khac
```

```
bgt t1, t3, check_lower # neu > 90 kiem tra tiep chu thuong
```

```
# Neu la chu HOA -> cong them 32 de thanh chu thuong
```

```
addi t1, t1, 32
```

```
sb t1, 0(t0)    # luu lai
```

```
j next_char
```

```
check_lower:
```

```
# Kiem tra neu la chu thuong (97 den 122)
```

```
li t2, 97      # a
```

```
li t3, 122     # z
```

```
blt t1, t2, next_char # Neu < 97, la ky tu khac -> Giu nguyen
```

```
bgt t1, t3, next_char # Neu > 122, la ky tu khac -> Giu nguyen
```

```
# Neu la chu thuong -> tru di 32
```

```
addi t1, t1, -32
```

```
sb t1, 0(t0)    # luu lai
```

```
next_char:
    addi t0, t0, 1    # chuyen sang ki tu tiep theo
    j loop
```

```
print_result:
    # print kq
    li a7, 4
    la a0, res_msg
    ecall
```

```
li a7, 4
la a0, buffer    # xau da doi
ecall
```

```
li a7, 10        # Thoat
ecall
```

Mô tả thuật toán sử dụng:

- **Mối liên hệ giữa chữ Hoa và chữ Thường:**
 - Trong bảng mã ASCII, chữ cái thường luôn cách chữ cái hoa tương ứng đúng **32** đơn vị.
 - Ví dụ: 'A' là 65, 'a' là 97 ($65 + 32 = 97$) nên ta chỉ cần lấy mã chữ hoa +32 là ra chữ thường và ngược lại.
- **Cách các thanh ghi thay đổi:**

- **t0**: Đóng vai trò trở vào vị trí ký tự trong RAM.
- **t1**: Lấy ký tự từ RAM lên để kiểm tra. Nếu thỏa mãn điều kiện là chữ cái, nó sẽ bị thay đổi giá trị (cộng hoặc trừ 32) rồi ghi đè ngược lại vào RAM bằng lệnh sb (Store Byte).

***Các giá trị ban đầu và khởi tạo:**

- t0 và t1 chưa được khởi tạo:

t0	5	0x00000000
t1	6	0x00000000

-nhập chuỗi theo mẫu: “xIn chAO 2023”

Nhap xau ky tu: xIn chAO2023

-t0 trở vào địa chỉ của kí tự đầu tiên:

t0	5	268501031
----	---	-----------

	Value (+0)	Value (+4)	Value (+8)	Value (+c)	Value (+10)	Value (+14)	Value (+18)	Value (+1c)
0000	p a h N	u a x	y k	: u t	t e K \0	a u q	u a s	i h k
0020	i o d	x \0 :	c n I	2 0 A h	\n 3 2 0	\0 \0 \0 \0	\0 \0 \0 \0	\0 \0 \0 \0
0040	\0 \0 \0 \0	\0 \0 \0 \0	\0 \0 \0 \0	\0 \0 \0 \0	\0 \0 \0 \0	\0 \0 \0 \0	\0 \0 \0 \0	\0 \0 \0 \0
0060	\0 \0 \0 \0	\0 \0 \0 \0	\0 \0 \0 \0	\0 \0 \0 \0	\0 \0 \0 \0	\0 \0 \0 \0	\0 \0 \0 \0	\0 \0 \0 \0
0080	\0 \0 \0 \0	\0 \0 \0 \0	\0 \0 \0 \0	\0 \0 \0 \0	\0 \0 \0 \0	\0 \0 \0 \0	\0 \0 \0 \0	\0 \0 \0 \0
00a0	\0 \0 \0 \0	\0 \0 \0 \0	\0 \0 \0 \0	\0 \0 \0 \0	\0 \0 \0 \0	\0 \0 \0 \0	\0 \0 \0 \0	\0 \0 \0 \0
00c0	\0 \0 \0 \0	\0 \0 \0 \0	\0 \0 \0 \0	\0 \0 \0 \0	\0 \0 \0 \0	\0 \0 \0 \0	\0 \0 \0 \0	\0 \0 \0 \0
00e0	\0 \0 \0 \0	\0 \0 \0 \0	\0 \0 \0 \0	\0 \0 \0 \0	\0 \0 \0 \0	\0 \0 \0 \0	\0 \0 \0 \0	\0 \0 \0 \0
0100	\0 \0 \0 \0	\0 \0 \0 \0	\0 \0 \0 \0	\0 \0 \0 \0	\0 \0 \0 \0	\0 \0 \0 \0	\0 \0 \0 \0	\0 \0 \0 \0
0120	\0 \0 \0 \0	\0 \0 \0 \0	\0 \0 \0 \0	\0 \0 \0 \0	\0 \0 \0 \0	\0 \0 \0 \0	\0 \0 \0 \0	\0 \0 \0 \0
0140	\0 \0 \0 \0	\0 \0 \0 \0	\0 \0 \0 \0	\0 \0 \0 \0	\0 \0 \0 \0	\0 \0 \0 \0	\0 \0 \0 \0	\0 \0 \0 \0
0160	\0 \0 \0 \0	\0 \0 \0 \0	\0 \0 \0 \0	\0 \0 \0 \0	\0 \0 \0 \0	\0 \0 \0 \0	\0 \0 \0 \0	\0 \0 \0 \0
0180	\0 \0 \0 \0	\0 \0 \0 \0	\0 \0 \0 \0	\0 \0 \0 \0	\0 \0 \0 \0	\0 \0 \0 \0	\0 \0 \0 \0	\0 \0 \0 \0
01a0	\0 \0 \0 \0	\0 \0 \0 \0	\0 \0 \0 \0	\0 \0 \0 \0	\0 \0 \0 \0	\0 \0 \0 \0	\0 \0 \0 \0	\0 \0 \0 \0

***Vòng lặp:**

-Lần 1:

-t1 được nạp giá trị nơi t0 trở vào:

t1	6	120
----	---	-----

-do t1 là chữ thường nên được chuyển sang chữ hoa(-32):

t0	5	268501031
t1	6	88

-Lần 2:

-t0 trở đến vị trí kí tự thứ 2:

t0	5	268501032
----	---	-----------

-t1 nhận giá trị mới(I):

t1	6	73
----	---	----

-t1 là chữ hoa nên mã accii được cộng thêm 32:

t1	6	105
----	---	-----

-Lần 3:

-t0 trở đến kí tự thứ 3:

t0	5	268501033
----	---	-----------

-t1 nhận giá trị mới(n):

t1	6	110
----	---	-----

-t1 chuyển sang hoa(-32):

t1	6	78
----	---	----

-Lần 4:

-t0:

t0	5	268501034
----	---	-----------

-t1(khoảng trắng) nên giữ nguyên.:

t1	6	32
----	---	----

Tương tự lặp đến hết chuỗi ta thu được chuỗi mới:

```
Ket qua sau khi doi: XiN CHao2023
```

[illegible]

